Target Machine Specification

2026-1, SWPP

Update Log

2026-05-21

- Stack area has been shifted to address [1600, 104000). `sp` register is initialized to 104000.

Input Program

Structure.
- The source program consists of a single IR file; There is no linking.
- The IR file has a main function, and may consist of more functions.
- A source program only uses i1, i8, i16, i32, i64, array types, and a pointer type.

Function.
- A function can have at most 16 arguments.
- There is no function attribute (e.g. read-only).
- `main()` is never called recursively.

Standard I/O.
- A source program takes input through `read()` calls. `read()` reads an integer and returns it as an i64 value.
- The output of the program is done via `write(i64)` calls. It writes the output as an unsigned integer in a new line.
- `read()` / `write(i64)` calls are connected to the standard input/output.

Misc.
- The test programs will never raise out-of-memory or stack overflow with the given inputs when compiled with the project skeleton.

Target Architecture

1. Architecture Overview

The architecture consists of a single-core CPU and a 64-bit memory space.

1.1 Register

- There are 32 64-bit general registers. They are named `r1`, `r2`, .., `r32`.
- There is a 64-bit stack pointer register, named sp.
- `r1`, `r2`, .., `r32` are initialized to 0, and `sp` is initialized to 104000.
- Registers can be assigned multiple times (it is not SSA).
- There are 16 64-bit read-only registers `arg1`,..,`arg16` for passing function arguments.

1.2 Memory

Loads and stores.
- The memory is accessed via `load`/`store`/`aload` instructions with 64-bit pointers.
- The exact formula for the cost calculation will be described below.
- An asynchronous memory load can be performed using `aload` (asynchronous load)

Stack.
- The stack area starts from address 104000, grows downward (-) down to 1600, and initialized to 0 at the beginning of execution.
- `sp` stores the base address of the current function stack, but you won't be able to access nor modify the value stored in this register.

Heap.
- The heap area starts from address 204800 and grows upward (+).
- Accessing an unallocated heap raises an error.
- Accessing the memory of address [0, 1600), [104000, 204800) raises an error.

Global Variables.
- Syntactically, there is no difference between global variables and heap-allocated blocks.
- The project skeleton lowers a global variable to a heap allocation (malloc call) at the beginning of the main(). So, they are placed at the beginning of the heap area.

1.3 Calling Convention

- When a `call` instruction is executed,
    - `r1..r32, sp, arg1..arg16` registers are automatically saved (you don't need to manually spill them).
    - Values of the arguments are automatically assigned to the registers `arg1`..`arg16`.
    - Values in registers do not change (r1..r32 and arg1..arg16, except registers used for argument passing)
- After the call returns, register values are automatically restored to the context before the call.

1.4 Cost

- The execution cost of a program can be calculated as 'program-wide instruction execution cost + maximum heap memory usage (in bytes)' * 1024.
- The code size is irrelevant to the total cost.

Memory usage cost.

- The memory usage cost is 1024 times the maximum heap-allocated byte size at any moment.
- For example, the memory usage cost of

```
r1 = malloc 8
free r1
r2 = malloc 8
free r2
```

is 1024 * 8 = 8192, because the maximum memory usage is 8 bytes.

Compile time.

- Compile time should be less than 1 minute.

2. Function & Basic Block

2.1 Function

Syntax:

```
start <funcname> <Narg>:
  … (basic blocks)
end <funcname>
```

- A function contains one or more basic blocks.
- `<funcname>` is a non-empty string, starting with an alphabet (a-zA-Z) or underscore(_), and consists of alphabets(a-zA-Z), underscore(_), digits(0-9), or dot(.).
- `<Narg>` describes the number of arguments.
- A function's return type is always i64.
- There is no variadic function.
- There is no nested function.

2.2 Basic Block

Syntax:

```
<bbname>:
  … (instructions)
```

- A basic block consists of one or more instructions.
- A basic block must end with a terminator instruction (see below for more details)
- `<bbname>` is a non-empty string, starting with a dot(.) and consists of alphabets(a-zA-Z), underscore(_), digits(0-9), or dot(.).

2.3 Comment

Syntax:

```
; <comment>
```

- A comment starts with a semicolon(;).

3. Instruction Set

Syntax:
- `<reg*>` stands for a general register (either r1..r32 or sp)
- `<cst*>` stands for a nonnegative integer constant in decimal (< 2^64)
- `<val*>` stands for a register (r1..r32, sp, or arg1..arg16) or a nonnegative integer constant
- Argument registers (e.g. `arg1`) cannot be assigned

Typically, a line of assembly looks like below:

```
op_name <val1> .. <valN>
<reg> = op_name <val1> .. <valN>
```

3.1 Control Flow

Kind	**Syntax**	**Cost**
Return Value	`ret <val>`	1
Unconditional Branch	`br <bb>`	30 forward jump is slow
Conditional Branch	`br <val_cond> <bb_true> <bb_false>`	90 for `bb_true` 30 for `bb_false` forward jump is slow
Switch Instruction	`switch <val_cond> <cst1> <bb1> .. <cstN> <bbN> <bb_default>`	60 forward jump is slow
Function call	`call <fn> <val1> .. <valN>` `<reg_ret> = call <fn> <val1> .. <valN>`	30

- `ret, br, switch` instructions can appear only at the end of a basic block.
- `<bb*>` is the name of a basic block to jump to.
- `<fn>` is the name of a function to call.
- `br / switch` cannot jump to a block in another function.

Forward Jump
When jumping to the basic block that appears after the jump instruction, the jump costs an additional 50%.

3.2 Heap Memory Allocation/Deallocation

Kind	Syntax	Cost
Heap Allocation	`<reg_ptr> = malloc <val_size>`	150
Heap Deallocation	`free <val_ptr>`	150

Malloc
- The size of `malloc` should be non-zero & a multiple of 8.
- The returned address by `malloc` is a multiple of 8.
- Allocated memory is initialized to 0, and heat is reset to 0 as well.

Free
- `free` deallocates a space associated with the given pointer.
- The pointer passed to `free` should point to the beginning of allocated heap space.

3.3 Memory Access

Kind	Syntax	Base Cost
Load	`<reg> = load <sz> <val_ptr>` `<sz> := 1|2|4|8`	Stack area: 30 Heap area: 50 Heat Memory
Store	`store <sz> <val> <val_ptr>` `<sz> := 1|2|4|8`	Stack area: 30 Heap area: 50 Heat Memory
Async Load	`<reg> = aload <sz> <val_ptr>` `<sz> := 1|2|4|8`	Stack area: 1 Heap area: 1 Cost to resolve Stack area: 36 Heap area: 60 Heat Memory (Gains heat when the instruction is called)

- Value in `<val_ptr>` should be multiple of `<sz>`. Unaligned memory access will crash the interpreter.

Load

- The `load` instruction reads the data at [`<val_ptr>` , `<val_ptr>+<sz>`), zero-extends it to 64 bits, and returns it.
- The memory is *little-endian*. The least significant byte of the value read by `load` is from `<val_ptr>`, and the most significant byte is from `<val_ptr>+<sz>-1`.

Store

- The `store` instruction truncates the value `<val>` to an `<sz>`*8-bit integer and writes it at [`<val_ptr>`, `<val_ptr>+<sz>`).

Asynchronous Load.

- The `aload` instruction behaves exactly the same as an ordinary load, except that one should wait until the loaded value is resolved to use it.

```
r2 = aload 8 r1
r3 = add r2 1 64    ; waits for cost 36 (stack) or 60 (heap)
```

- After executing an `aload` instruction, it takes the cost n = 36 for stack area and n = 60 for heap area for the returning register to be y. e.g.,

```
r2 = aload 8 r1  ; suppose r1 contains an address to heap area
...              ; instructions here take cost m (no use of r2)
call write r2    ; waits for 60 - m
```

  Total cost = 1 (for `aload`) + m + 3 (for `call`) + max(0, 60 - m) (for waiting)

- Access to addresses that overlap an unresolved async load is allowed. e.g.,

```
store 8 3 r1
r2 = aload 8 r1
store 8 42 r1
r3 = load 8 r1    ; loads 42
call write r2     ; prints 3 after the loaded value (r2) is resolved
```

- Overwriting a register that is waiting for an async load is allowed. e.g.,

```
store 8 3 r1      ; suppose r1 contains an address to stack area
r2 = aload 8 r1
r2 = add 42 0 64  ; does not wait for aload to be resolved
call write r2     ; prints 42
```

  Total cost = 20 (for `store`) + 1 (for `aload`) + 5 (for `add`) + 3 (for `call`)

Heat Memory

- Whenever a heap memory is accessed (load/store/aload), a part of memory accumulates some amount of heat.
- Every consecutive 8 bytes of heap memory shares a heat segment. Whenever an access is made to the segment, the segment gains 400 heat points (HP).
- Each segment can accumulate only up to 2000HP. Excessive heat will be ‘dissipated’.
- Wider memory access adds heat to nearby segments as well. Refer to the table below.

⬇️	Accessed segment
	Always accumulates 400 HP
	Accumulates 400 HP when access width >= 2B
	Accumulates 400 HP when access width >= 4B
	Accumulates 400 HP when access width is 8B

- Remaining heat in the heat segment increases the access cost by (HP / 10) floor.
- Heat segments cool down when they're not accessed. Every instruction except access to that heat segment reduces the HP by its execution cost.
- Heat segments from two different mallocs are considered disjoint, and never add heat via memory access even if they are adjacent address-wise.

3.4 Scalar Arithmetic

Kind	**Name**	**Cost**

Integer Multiplication/Division	`<reg> = udiv <val1> <val2> <bw>`
	`<reg> = sdiv <val1> <val2> <bw>`
	`<reg> = urem <val1> <val2> <bw>`
	`<reg> = srem <val1> <val2> <bw>`
	`<reg> = mul  <val1> <val2> <bw>`
	`<bw> := 1|8|16|32|64`
		2
		
		If followed by add/sub instruction, cost reduced to 0

Integer Shift
- shl: shift-left
- lshr: logical shift-right
- ashr: arithmetic shift-right
	`<reg> = shl  <val1> <val2> <bw>`
	`<reg> = lshr <val1> <val2> <bw>`
	`<reg> = ashr <val1> <val2> <bw>`
	`<bw> := 1|8|16|32|64`
		10

Bitwise Operation	`<reg> = and  <val1> <val2> <bw>`
	`<reg> = or   <val1> <val2> <bw>`
	`<reg> = xor  <val1> <val2> <bw>`
	`<bw> := 1|8|16|32|64`
		10

Integer Add/Sub
- (e/o)add: addition optimized for even/odd result, respectively
- (e/o)sub: subtraction optimized for even/odd result, respectively
	`<reg> = eadd <val1> <val2> <bw>`
	`<reg> = oadd <val1> <val2> <bw>`
	`<reg> = esub <val1> <val2> <bw>`
	`<reg> = osub <val1> <val2> <bw>`
	`<bw> := 1|8|16|32|64`
		10 if the parity matches the expectation, 20 otherwise

Comparison - `<cond>` is equivalent to the `cond` of LLVM IR's icmp	`<reg> = icmp <cond> <val1> <val2> <bw>` `<bw> := 1|8|16|32|64`	3
Ternary operation	`<reg> = select <val_cond> <val_true> <val_false>`	3
Assertion An assertion fail is an error. Used for testing	`assert_eq <val1> <val2>`	0

- The signed division and remainder operations truncates the quotient towards 0, following LLVM IR. See https://en.wikipedia.org/wiki/Modulo.
- For integer arithmetic and comparison operations, `<bw>` is the size of bitwidth of inputs that the operation assumes. For example, `` `ashr 511 2 8` `` takes the lowest 8-bits from inputs (which is 255 = -1), performs arithmetic right shift, and zero-extends it to 64 bits. So, its result is 255.
- For select, if <val_cond> is not one of 0 or 1, the interpreter will crash.

4. 4-Phobia

The target machine has irrational fear towards number `4`. Any instruction that involves constant integer `4` will result in extra 10 costs.
- Even if `4` appears more than one time in an instruction, cost is added only once.
- When a multiplication is followed by add/sub, but involves `4`, it will still suffer from the phobia and cost 10.
- `assert_eq` is exempt from the phobia.